

MANTIS: Metacognitive Adaptive Network with Tiered Inference Strategies for Hallucination Mitigation

Anonymous Authors
Institution Name
email@institution.edu

Abstract

Large Language Models (LLMs) exhibit a fundamental trade-off between long-context retention, computational efficiency, and hallucination rates. Existing approaches either scale quadratically with context length (dense transformers), compress information lossy (state-space models), or lack contextual statefulness (retrieval-augmented generation). We propose **MANTIS** (Metacognitive Adaptive Network with Tiered Inference Strategies), a hybrid architecture that couples a sparse Mixture-of-Experts (MoE) backbone with a meta-controlled three-tier memory system. We present a complete reference implementation in which a lightweight *Meta-Controller*, trained with reinforcement learning (PPO), routes each query between (1) immediate self-attention, (2) episodic state-space compression (Mamba-based SSM), and (3) semantic vector retrieval (FAISS), and biases the MoE’s expert selection. An optional critic model verifies outputs and triggers abstention when it doubts them. This paper describes the architecture and its four-stage training pipeline as a framework for future research into adaptive memory systems. No model has been trained yet, so we discuss the architecture’s theoretical properties and the evaluation needed to test its claims on long-context benchmarks.

1 Introduction

The success of Large Language Models [1, 2, 3] has been accompanied by persistent challenges in long-context reasoning and factual accuracy. Current architectures face three fundamental limitations:

(1) **Quadratic Scaling:** Self-attention mechanisms in transformers scale as $O(L^2)$ with sequence length L , imposing prohibitive computational costs for contexts exceeding 8K tokens [4, 5].

(2) **Lost-in-the-Middle:** Dense attention struggles to maintain uniform retrieval accuracy across extended contexts, exhibiting U-shaped performance curves where mid-sequence information is preferentially forgotten [6].

(3) **Hallucination Persistence:** Despite advances in pre-training scale and alignment, LLMs continue to generate plausible but factually incorrect outputs, particularly when forced to extrapolate beyond training distributions [7, 8].

Existing mitigation strategies exhibit complementary weaknesses. *Retrieval-Augmented Generation* (RAG) [9] offloads long-term memory to external datastores but lacks stateful compression of conversation context. *State Space Models* (SSMs) [10, 11] achieve linear complexity but sacrifice the ability to perform targeted retrieval over historical context. *Sparse attention* patterns [12, 13] reduce compute but maintain quadratic worst-case complexity.

1.1 Contributions

We propose MANTIS, a metacognitive architecture that integrates three memory tiers under unified RL-based control:

1. **Sparse MoE Backbone:** A base configuration with 6.8B parameters, 8 experts and top-2 routing, activating about 2B parameters per token.
2. **Three-Tier Memory Hierarchy:**
 - *L1 (Working Memory):* Causal self-attention with rotary position embeddings over an 8K-token window.
 - *L2 (Episodic Memory):* Stacked Mamba SSM blocks that compress each recent interaction (up to 8K tokens) into a 256-dimensional state.

- *L3 (Semantic Memory)*: A FAISS vector index with sub-linear retrieval over 1M+ historical entries.
3. **Meta-Controller Architecture**: A 105M-parameter residual MLP policy, trained with PPO, that routes queries based on the query embedding, model uncertainty and memory state.
 4. **Integrated Critic Model**: A 155M-parameter encoder that scores responses against retrieved evidence and triggers abstention.

We describe the system architecture and its implementation, and analyze its scaling properties and expected benefits over dense baselines.

2 Related Work

Sparse Models & Mixture of Experts. Switch Transformer [14] and Mixtral [15] demonstrate that sparse expert activation maintains model quality while reducing per-token compute. Our MoE backbone follows top-K routing principles but integrates with memory-augmented components rather than serving as the sole architecture.

State Space Models. Structured SSMs (S4 [10], Mamba [11], Jamba [16]) achieve linear complexity through selective state compression. While Jamba explores SSM-attention hybrids at the layer level, MANTIS implements metacognitive routing with hierarchical memory and explicit control over inference strategies.

Memory-Augmented Networks. Neural Turing Machines [17] and Memory Networks [18] pioneered external memory interfaces. Modern RAG systems [9, 19] combine dense retrievers with generative models. MANTIS extends this paradigm with tiered inference strategies and learned metacognitive access policies.

Hallucination Detection. Recent work addresses factual errors through self-consistency checking [20], retrieval verification [21], and uncertainty quantification [22]. Our critic model explicitly conditions on retrieved evidence for calibrated confidence estimation.

3 Architecture

3.1 Notation and Problem Setup

Given an input sequence $\mathbf{x} = (x_1, \dots, x_L)$ where $x_i \in \mathcal{V}$ (vocabulary), we seek to model the conditional distribution $p(x_{t+1} | x_{\leq t})$ while maintaining three objectives:

$$\text{Minimize: } \mathcal{L}_{LM} = - \sum_{t=1}^L \log p(x_t | x_{<t}) \quad (\text{Language Modeling}) \quad (1)$$

$$\text{Minimize: } \mathcal{C}_{compute}(x) \quad (\text{Computational Cost}) \quad (2)$$

$$\text{Maximize: } \mathcal{Q}_{factual}(x) \quad (\text{Factual Accuracy}) \quad (3)$$

Let d_{model} denote embedding dimension, $N = 8$ the number of experts, and $K = 2$ the activation sparsity.

3.2 Backbone

The base model is a decoder-only transformer with pre-norm LayerNorm blocks. Attention uses rotary position embeddings [27] and fused scaled dot-product attention over a window of L_{max} tokens, with a key/value cache for generation. Input embeddings and the output projection share weights. Every block’s feed-forward sublayer is the MoE layer below; with $N = 1$ it reduces to a dense feed-forward network. Table 1 lists the implemented configurations.

Tokenizer. The implementation uses a 512-token vocabulary matched longest-first through a trie. It holds 283 domain tokens for the text protocol of a synthetic ecological simulator, which the project uses as its first training corpus, plus 18 extra ASCII characters, 161 UTF-8 byte-fallback tokens and 50 reserved slots. Numbers are split into digits. Any string round-trips losslessly, but natural-language text falls back to single characters and bytes, so its sequences are several times longer than under a subword vocabulary.

3.3 Base Mixture-of-Experts Layer

Each MoE layer consists of N expert networks $\{E_i\}_{i=1}^N$ where $E_i : \mathbb{R}^{d_{model}} \rightarrow \mathbb{R}^{d_{model}}$ is a two-layer feed-forward network:

$$E_i(\mathbf{x}) = W_{2,i} \text{GELU}(W_{1,i}\mathbf{x} + \mathbf{b}_{1,i}) + \mathbf{b}_{2,i} \quad (4)$$

Table 1: Model presets in the reference implementation (vocabulary of 512 tokens, $L_{max} = 8192$).

Preset	d_{model}	Layers	Experts	Parameters	Active
micro	256	4	1 (dense)	3M	3M
tiny	512	6	4	57M	32M
small	1024	12	4	454M	252M
base	2048	24	8	6.85B	2.02B

where GELU is the Gaussian Error Linear Unit activation [23].

The gating network $G(\mathbf{x})$ computes routing probabilities:

$$\mathbf{h} = W_g \mathbf{x} + \mathbf{b}_g + \beta \in \mathbb{R}^N \quad (5)$$

$$\mathbf{p} = \text{Softmax}(\mathbf{h}) \quad (6)$$

where β is the expert bias chosen by the Meta-Controller for the whole query (zero when it does not intervene). The bias shifts the learned gate logits rather than replacing them, so learned routing patterns survive.

Top- K routing selects the expert subset $\mathcal{T} = \text{TopK}(\mathbf{p}, K = 2)$ and renormalizes:

$$\mathbf{y} = \sum_{i \in \mathcal{T}} \frac{p_i}{\sum_{j \in \mathcal{T}} p_j} E_i(\mathbf{x}) \quad (7)$$

Load Balancing. To prevent expert collapse, each MoE layer adds the Switch Transformer auxiliary loss [14]:

$$\mathcal{L}_{bal} = \alpha \cdot N \sum_{i=1}^N f_i \cdot \bar{p}_i \quad (8)$$

where f_i is the fraction of tokens in the batch whose top-1 expert is i , $\bar{p}_i = \frac{1}{|B|} \sum_{x \in B} p_i(x)$ is the average routing probability across the batch, and $\alpha = 0.01$. The model sums $\mathcal{L}_{bal}$ over its layers.

3.4 Meta-Controller

The Meta-Controller π_θ sees one vector per query, so it is a residual MLP rather than a sequence model: 6 pre-norm feed-forward blocks (inner width 4096; 105M parameters at $d_{model} = 2048$). The base model first runs a forward pass over the query. Its mean-pooled final hidden state is the query embedding $\mathbf{e}_q$, and its next-token distributions yield the features of the state summary:

$$\mathbf{s}_t = \text{Enc}_\psi(u_t, \ell_t, \mathbf{m}_t, c_t) \quad (9)$$

where:

- u_t : mean next-token entropy over the query, normalized by $\log |\mathcal{V}|$
- ℓ_t : query length relative to the attention window
- $\mathbf{m}_t$: episodic fill (entries over capacity) and whether semantic memory holds any entry
- c_t : mean top-1 next-token probability (confidence)

Enc_ψ is a small learned encoder that projects these features to a 128-dimensional summary.

The controller computes $\mathbf{z}_t = \text{MLP}_\theta([\mathbf{e}_q; \mathbf{s}_t])$ and outputs four gate probabilities and an expert bias:

$$g_{exit} = \sigma(W_{exit} \mathbf{z}_t) \quad (\text{Early Exit}) \quad (10)$$

$$g_{epi} = \sigma(W_{epi} \mathbf{z}_t) \quad (\text{Episodic Access}) \quad (11)$$

$$g_{sem} = \sigma(W_{sem} \mathbf{z}_t) \quad (\text{Semantic Access}) \quad (12)$$

$$g_{ver} = \sigma(W_{ver} \mathbf{z}_t) \quad (\text{Verification}) \quad (13)$$

$$\mu_{exp} = W_{exp} \mathbf{z}_t \quad (\text{Expert Bias, raw logits}) \quad (14)$$

where σ is the sigmoid function.

Stochastic policy. During RL, each gate is a Bernoulli variable with probability g_* , and the expert bias is drawn from $\mathcal{N}(\mu_{exp}, \text{diag}(\sigma^2))$ with a learned σ (initialized to 0.5). At inference the policy is deterministic: $a_* = \mathbb{I}[g_* > \tau]$ with $\tau = 0.5$, and $\beta = \mu_{exp}$. A gate whose component is missing (no episodic memory, semantic store or critic) stays closed and contributes nothing to the policy log-probability.

Routing. The early-exit path requires both an open exit gate and low uncertainty ($u_t < 0.2$). It decodes the query alone, with no memory and no expert bias. Otherwise the open memory gates retrieve context, the retrieved text is prepended to the query within the attention budget, and decoding applies the expert bias β in every MoE layer. Verification runs only on this full path.

3.5 Episodic Memory: Selective State Space Model

The L2 episodic memory implements a selective SSM following the Mamba architecture [11]. For input sequence $\{\mathbf{x}_t\}$, the continuous-time state-space equations are:

$$\mathbf{h}'(t) = \mathbf{A}\mathbf{h}(t) + \mathbf{B}\mathbf{x}(t) \quad (15)$$

$$\mathbf{y}(t) = \mathbf{C}\mathbf{h}(t) \quad (16)$$

Selective Discretization. We compute input-dependent time-scales:

$$\Delta_t = \text{Softplus}(W_\Delta \mathbf{x}_t + \mathbf{b}_\Delta) \quad (17)$$

The continuous parameters are discretized via zero-order hold:

$$\bar{\mathbf{A}}_t = \exp(\Delta_t \mathbf{A}) \quad (18)$$

$$\bar{\mathbf{B}}_t = (\Delta_t \mathbf{A})^{-1}(\exp(\Delta_t \mathbf{A}) - \mathbf{I})\Delta_t \mathbf{B} \quad (19)$$

The discrete recurrence becomes:

$$\mathbf{h}_t = \bar{\mathbf{A}}_t \mathbf{h}_{t-1} + \bar{\mathbf{B}}_t \mathbf{x}_t \quad (20)$$

$$\mathbf{y}_t = \mathbf{C}_t \mathbf{h}_t + \mathbf{D}\mathbf{x}_t \quad (21)$$

where $\mathbf{B}_t = W_B \mathbf{x}_t$ and $\mathbf{C}_t = W_C \mathbf{x}_t$ provide input-dependent selectivity.

Implementation. The module stacks 8 Mamba blocks (state size 256, expansion 2, convolution width 4) over the base model’s hidden states. Mean pooling and a linear projection reduce each interaction of up to 8K tokens to a 256-dimensional state, so every stored entry has an $O(1)$ retrieval key. The engine writes every answered query together with its response. The buffer keeps the latest 100 interactions with their token IDs; retrieval ranks them by cosine similarity between the query’s state and the stored states and returns the top 3 as decoded text.

3.6 Semantic Memory: Vector Index Retrieval

The L3 semantic memory uses a FAISS index [24] in which every vector keeps a stable ID:

- **Embedding:** Each entry is the mean-pooled base-model embedding of its text, passed through a linear projection P (trained in Stage 2) and L2-normalized, so L2 ranking equals cosine ranking.
- **Indexing:** Below 10K entries the index searches exhaustively. From 10K entries it trains an inverted file (IVF) over $n_{list} = \min(4\sqrt{M}, M/39)$ Voronoi cells and probes $\min(n_{list}, 16)$ of them per query.
- **Quantization:** Product quantization with 64 sub-quantizers of 8 bits compresses each 2048-dimensional vector into 64 bytes.
- **Eviction:** Beyond capacity the oldest entries are evicted. Evicted IDs become tombstones, and the index is rebuilt once more than 20% of it is stale.

For query $\mathbf{q}$, retrieval proceeds as:

$$\mathcal{R}_k(\mathbf{q}) = \text{TopK}_{\mathbf{v} \in \mathcal{I}} \left(\frac{(P\mathbf{q})^\top (P\mathbf{v})}{\|P\mathbf{q}\| \|P\mathbf{v}\|} \right) \quad (22)$$

with $k = 5$ at inference. Scanning n_{list} centroids and 16 cells of about M/n_{list} vectors each costs $O(\sqrt{M})$.

Background Consolidation. Every hour, when L2 holds at least 5 entries, a consolidator moves important interactions to L3. The importance of entry e is the mean norm of its hidden states $\mathbf{x}_1, \dots, \mathbf{x}_T$, min-max normalized across the buffer:

$$I(e) = \frac{n(e) - \min_{e'} n(e')}{\max_{e'} n(e') - \min_{e'} n(e')}, \quad n(e) = \frac{1}{T} \sum_{t=1}^T \|\mathbf{x}_t\|_2 \quad (23)$$

Entries with $I(e) \geq 0.7$ are grouped greedily by cosine similarity (≥ 0.8) of their SSM states. Each group is represented by the decoded text of its longest interaction, embedded like a query, and written to L3. Only entries whose write succeeded leave L2.

3.7 Critic Model for Hallucination Detection

The critic C_ϕ is a 12-layer pre-norm transformer encoder ($d_{model} = 1024$, 16 heads; 155M parameters) that takes the query $\mathbf{q}$, generated response $\mathbf{r}$, and retrieved facts $\mathcal{F}$ as input:

$$P_{ver}, C_{ver} = C_\phi([\mathbf{q}; \mathbf{r}; \mathcal{F}]) \quad (24)$$

where $P_{ver} \in [0, 1]$ is the correctness probability and $C_{ver} \in [0, 1]$ is the confidence score. Learned segment embeddings mark the three parts, and the input fits a 2048-token budget: the last 512 query tokens, the first 512 fact tokens, and as much of the response as remains. Mean pooling feeds two sigmoid heads. When $P_{ver} < 0.6$, the engine replaces the response with an abstention.

Training minimizes the calibrated loss:

$$\mathcal{L}_{critic} = \text{BCE}(P_{ver}, y_{true}) + \lambda \|C_{ver} - y_{true}\|_2^2 \quad (25)$$

with $\lambda = 0.5$ for balanced calibration.

4 Proposed Training Methodology

Training runs in four stages. Stage 1 is required; Stages 2 and 4 each need only the Stage 1 model, and Stage 3 enables whichever gates the Stage 2 and Stage 4 outputs support.

4.1 Stage 1: Pre-training

The base MoE model is trained alone on language modeling:

$$\mathcal{L}_{total} = \mathcal{L}_{LM} + \sum_{\text{layers}} \mathcal{L}_{bal} \quad (26)$$

The implementation uses AdamW [25] ($\beta = (0.9, 0.95)$, weight decay 0.01) with a peak learning rate of 3×10^{-4} , linear warmup, cosine decay to 10% of the peak, and FP16 mixed precision. 8-bit AdamW, gradient checkpointing and DeepSpeed ZeRO-2 are optional. Documents end with an EOS token and are packed into windows of $L + 1$ tokens. For natural-language pre-training we propose a corpus of about 2T tokens from web text, books and code.

4.2 Stage 2: Memory Fine-Tuning

With the base model frozen, Stage 2 trains the episodic SSM and the semantic projection P on (query, context) pairs. Both use InfoNCE over in-batch negatives with temperature 0.07: the SSM state of a query must match the state of its context, and the projected embedding of a query must retrieve its context. The losses are weighted 1.0 (episodic) and 0.5 (semantic), with learning rate 10^{-5} . Afterwards every context is written to an L3 store for Stage 3 and inference.

4.3 Stage 3: Meta-Controller Reinforcement Learning

The Meta-Controller and the state encoder are optimized with Proximal Policy Optimization (PPO) [26]. Each query is one episode, so the problem is a contextual bandit and the objective is the expected reward of a single routing decision:

$$J(\theta) = \mathbb{E}_{q, a \sim \pi_\theta(\cdot|q)} [R(q, a)] \quad (27)$$

Reward Function:

$$R = w_1 r_{acc} - w_2 r_{lat} - w_3 r_{comp} + w_4 r_{calib} \quad (28)$$

where:

- r_{acc} : +1 if the reference answer appears in the response, -1 otherwise
- r_{lat} : latency divided by a running mean of observed latency
- $r_{comp} = 0.5 \cdot \frac{\text{active_params}}{\text{total_params}} + 0.3 \cdot \mathbb{I}_{epi} + 0.2 \cdot \mathbb{I}_{sem}$: compute cost, where the indicators mark memory reads on the full path
- $r_{calib} = -(u_t - (1 - y_{true}))^2$: calibration, rewarding low uncertainty on correct answers and high uncertainty on wrong ones

Weights: $w_1 = 1.0, w_2 = 0.3, w_3 = 0.2, w_4 = 0.5$.

PPO Update: The policy is updated by minimizing the clipped surrogate loss:

$$\mathcal{L}^{PPO}(\theta) = -\mathbb{E} \left[\min \left(r(\theta) \hat{A}, \text{clip}(r(\theta), 1 - \epsilon, 1 + \epsilon) \hat{A} \right) \right] \quad (29)$$

where $r(\theta) = \frac{\pi_{\theta}(a|s)}{\pi_{\theta_{old}}(a|s)}$ is the probability ratio over the joint gate and expert-bias action, $\hat{A} = R - V_{\omega}(\mathbf{e}_q, \mathbf{s})$ is the advantage against a learned value network (normalized per batch), and $\epsilon = 0.2$. Each update runs 4 epochs over a batch of 256 on-policy episodes in minibatches of 64, with learning rate 10^{-5} , gradient clipping at 1.0 and a clipped value loss.

4.4 Stage 4: Critic Training

The critic is trained with supervision on labeled (query, response, facts) triples, using AdamW with learning rate 10^{-4} and the loss above. We propose curated hallucination datasets (HaluEval [31], FaithDial [32]) as training data, with:

- Positive examples: Factually correct responses with supporting evidence
- Negative examples: Plausible but incorrect outputs with contradictory evidence

5 Proposed Evaluation Protocol

5.1 Baselines

We propose comparing MANTIS against the following baselines to validate its effectiveness:

- **Llama-3-8B** [2]: Dense transformer baseline
- **Mixtral-8x7B** [15]: Sparse MoE without memory augmentation
- **Mamba-2.8B** [11]: Pure SSM architecture
- **Llama-3-8B + RAG**: Dense model with FAISS retrieval

5.2 Proposed Datasets

- **LongBench** [33]: Long-context QA (16K-128K tokens)
- **TruthfulQA** [34]: Hallucination-prone questions
- **HaluEval** [31]: Synthetic hallucination detection
- **NarrativeQA** [35]: Book-length comprehension

5.3 Proposed Metrics

1. **Accuracy:** Exact match / F1 score on QA tasks
2. **Hallucination Rate:** Fraction of factually incorrect generations
3. **Latency:** Wall-clock inference time (A100 GPU)
4. **FLOPs:** Floating-point operations per token
5. **Calibration Error:** Expected Calibration Error (ECE) [36]

5.4 Implemented Harness

The reference implementation already evaluates MMLU [28], TruthfulQA [34], HumanEval [29] and GSM8K [30], loaded from the HuggingFace Hub, for either the base model alone or the full engine rebuilt from a Stage 3 policy. A response’s confidence is the geometric-mean probability of its generated tokens. TruthfulQA counts a response as truthful when its token F1 is higher against a true reference than against every false one. HumanEval runs generated code in a resource-limited subprocess, which is not a security sandbox.

6 Expected Properties

6.1 Computational Efficiency

The hierarchical memory design is expected to enable graceful degradation: simple queries should bypass expensive retrieval (Gate 1), while complex reasoning activates the full memory hierarchy. We anticipate that RL optimization will learn query-specific routing policies, potentially achieving significant speedups compared to dense baselines by activating the minimal necessary parameters for each token.

6.2 Scaling Analysis

Theoretically, MANTIS should maintain sub-quadratic complexity as context length increases, due to the $O(1)$ memory footprint of the episodic memory and $O(\sqrt{M})$ retrieval complexity of the semantic memory. This contrasts with dense baselines which exhibit quadratic growth.

7 Discussion

7.1 Architectural Design Decisions

The decision to separate memory into three tiers is motivated by the distinct access patterns required for effective language modeling. Working memory handles immediate syntax and local coherence; episodic memory maintains a compressed narrative thread; and semantic memory provides access to long-term facts. The meta-controller serves as a learned arbitrator, aiming to balance the trade-off between retrieval cost and information gain.

7.2 Hallucination Mitigation Strategy

The critic model is designed to improve calibration by conditioning verification on retrieved evidence. By explicitly modeling the confidence of generated outputs against retrieved facts, the system aims to filter out ungrounded generation. The expected failure modes would likely involve multi-hop reasoning where the necessary evidence is distributed across different memory tiers, requiring complex routing decisions.

7.3 Limitations

- **No Trained Weights:** Every component ships untrained. The design claims above remain hypotheses until large-scale training and evaluation test them.
- **Training Complexity:** RL optimization of the meta-controller presents a non-trivial stability challenge and requires careful hyperparameter tuning. Its accuracy reward uses substring matching against a reference answer, which misjudges paraphrases.
- **Memory Maintenance:** Each L3 rebuild retrains the IVF-PQ codebooks over all live entries, a cost that grows faster than linearly in M and may become a bottleneck for very large stores. Semantic memory also keeps its full-precision vectors in RAM (more than 8GB for 1M entries at $d_{model} = 2048$).
- **Tokenizer Specialization:** The 512-token vocabulary targets the simulation protocol, so natural-language workloads pay for longer sequences.
- **Cold Start:** The system requires a warmup period to populate the episodic memory before it can effectively leverage historical context.

8 Conclusion

We have presented MANTIS, a proposed metacognitive adaptive network designed to address fundamental trade-offs in long-context LLM design. It combines sparse MoE computation with tiered memory under RL-based meta-control, in the hope of improving both efficiency and factual accuracy. This paper details the architecture, its four-stage training

pipeline and its reference implementation, which are ready for training and evaluation. If the design holds, learned routing policies could allocate compute according to query complexity.

Pending validation of the core architecture, future work could explore: (1) scaling memory capacity to 10M+ entries, (2) continual learning protocols for memory consolidation, and (3) integration with tool-augmented reasoning systems.

Broader Impact

The proposed MANTIS architecture aims to improve reliability in high-stakes applications (medical QA, legal reasoning). However, the critic model might introduce brittleness if trained on biased datasets, potentially amplifying systemic biases. Any future deployment should include human-in-the-loop verification for critical decisions.

References

- [1] T. Brown et al., “Language Models are Few-Shot Learners,” *NeurIPS*, 2020.
- [2] H. Touvron et al., “LLaMA: Open and Efficient Foundation Language Models,” *arXiv:2302.13971*, 2023.
- [3] Anthropic, “Claude 3 Model Family,” Technical Report, 2024.
- [4] A. Vaswani et al., “Attention is All You Need,” *NeurIPS*, 2017.
- [5] T. Dao et al., “FlashAttention: Fast and Memory-Efficient Exact Attention,” *NeurIPS*, 2022.
- [6] N. Liu et al., “Lost in the Middle: How Language Models Use Long Contexts,” *TACL*, 2023.
- [7] Y. Zhang et al., “Siren’s Song in the AI Ocean,” *EMNLP*, 2023.
- [8] Z. Ji et al., “Survey of Hallucination in Natural Language Generation,” *ACM Comp. Surveys*, 2023.
- [9] P. Lewis et al., “Retrieval-Augmented Generation for Knowledge-Intensive NLP,” *NeurIPS*, 2020.
- [10] A. Gu et al., “Efficiently Modeling Long Sequences with Structured State Spaces,” *ICLR*, 2022.
- [11] A. Gu and T. Dao, “Mamba: Linear-Time Sequence Modeling with Selective State Spaces,” *arXiv:2312.00752*, 2023.
- [12] R. Child et al., “Generating Long Sequences with Sparse Transformers,” *arXiv:1904.10509*, 2019.
- [13] I. Beltagy et al., “Longformer: The Long-Document Transformer,” *arXiv:2004.05150*, 2020.
- [14] W. Fedus et al., “Switch Transformers: Scaling to Trillion Parameter Models,” *JMLR*, 2022.
- [15] A. Q. Jiang et al., “Mixtral of Experts,” *arXiv:2401.04088*, 2024.
- [16] O. Lieber et al., “Jamba: A Hybrid Transformer-Mamba Language Model,” *arXiv:2403.19887*, 2024.
- [17] A. Graves et al., “Neural Turing Machines,” *arXiv:1410.5401*, 2014.
- [18] J. Weston et al., “Memory Networks,” *ICLR*, 2015.
- [19] G. Izacard et al., “Atlas: Few-shot Learning with Retrieval Augmented Language Models,” *arXiv:2208.03299*, 2022.
- [20] P. Manakul et al., “SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection,” *EMNLP*, 2023.
- [21] L. Gao et al., “RARR: Researching and Revising What Language Models Say,” *ACL*, 2023.
- [22] L. Kuhn et al., “Semantic Uncertainty: Linguistic Invariances for Uncertainty Estimation,” *ICLR*, 2023.
- [23] D. Hendrycks and K. Gimpel, “Gaussian Error Linear Units (GELUs),” *arXiv:1606.08415*, 2016.
- [24] J. Johnson et al., “Billion-scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, 2019.
- [25] I. Loshchilov and F. Hutter, “Decoupled Weight Decay Regularization,” *ICLR*, 2019.

- [26] J. Schulman et al., “Proximal Policy Optimization Algorithms,” *arXiv:1707.06347*, 2017.
- [27] J. Su et al., “RoFormer: Enhanced Transformer with Rotary Position Embedding,” *arXiv:2104.09864*, 2021.
- [28] D. Hendrycks et al., “Measuring Massive Multitask Language Understanding,” *ICLR*, 2021.
- [29] M. Chen et al., “Evaluating Large Language Models Trained on Code,” *arXiv:2107.03374*, 2021.
- [30] K. Cobbe et al., “Training Verifiers to Solve Math Word Problems,” *arXiv:2110.14168*, 2021.
- [31] Y. Li et al., “HaluEval: A Large-Scale Hallucination Evaluation Benchmark,” *EMNLP*, 2023.
- [32] N. Dziri et al., “FaithDial: A Faithful Benchmark for Information-Seeking Dialogue,” *TACL*, 2022.
- [33] Y. Bai et al., “LongBench: A Bilingual, Multitask Benchmark for Long Context Understanding,” *arXiv:2308.14508*, 2023.
- [34] S. Lin et al., “TruthfulQA: Measuring How Models Mimic Human Falsehoods,” *ACL*, 2022.
- [35] T. Kociský et al., “The NarrativeQA Reading Comprehension Challenge,” *TACL*, 2018.
- [36] C. Guo et al., “On Calibration of Modern Neural Networks,” *ICML*, 2017.